

Code Explanation

Delta Live Tables Pipeline for Customer Order Processing

This code implements a Delta Live Tables pipeline for processing customer and order data. It includes data cleaning, SCD Type 2 implementation for historical tracking, and aggregation of customer spending.

Overview

The pipeline reads customer and order data from CSV files, cleans and processes the data, and creates several tables to track customer and order information. It implements SCD Type 2 to maintain a history of changes to customer and order data.

Step 1: Define Schemas for Customer and Order Data

The first step is to define the schemas for the customer and order data. These schemas specify the structure of the data, including the column names and data types.

```
customer_schema = StructType([
    StructField("CustId", StringType(), True),
    StructField("Name", StringType(), True),
    StructField("EmailId", StringType(), True),
    StructField("Region", StringType(), True)
])

order_schema = StructType([
    StructField("OrderId", StringType(), True),
    StructField("ItemName", StringType(), True),
    StructField("PricePerUnit", DoubleType(), True),
    StructField("Qty", IntegerType(), True),
    StructField("Date", DateType(), True),
    StructField("CustId", StringType(), True)
])
```

Step 2: Read Source CSV Data

The next step is to read the customer and order data from CSV files using the defined schemas.

```
@dlt.table(
    name="bronze_customer",
    comment="Raw customer data from CSV files"
)
def bronze_customer():
    return (
        spark.read
        .option("header", "true")
        .schema(customer_schema)
        .csv(CUSTOMER_SOURCE_PATH)
    )

@dlt.table(
    name="bronze_order",
    comment="Raw order data from CSV files"
)
def bronze_order():
    return (
```

```

spark.read
.option("header", "true")
.schema(order_schema)
.csv(ORDER_SOURCE_PATH)
)

```

Step 3: Clean Customer Data

The customer data is cleaned by removing null values and duplicates.

```

@dlt.table(
  name="silver_customer",
  comment="Cleaned customer data with nulls and duplicates removed"
)
def silver_customer():
  return (
    dlt.read("bronze_customer")
    .filter(
      col("CustId").isNotNull() &
      col("Name").isNotNull() &
      col("EmailId").isNotNull() &
      col("Region").isNotNull()
    )
    .dropDuplicates(["CustId"])
  )

```

Step 4: Clean Order Data and Add TotalAmount Column

The order data is cleaned by removing null values and duplicates, and a new column `TotalAmount` is added by multiplying `PricePerUnit` and `Qty`.

```

@dlt.table(
  name="silver_order",
  comment="Cleaned order data with TotalAmount calculated"
)
def silver_order():
  return (
    dlt.read("bronze_order")
    .filter(
      col("OrderId").isNotNull() &
      col("ItemName").isNotNull() &
      col("PricePerUnit").isNotNull() &
      col("Qty").isNotNull() &
      col("Date").isNotNull() &
      col("CustId").isNotNull()
    )
    .dropDuplicates(["OrderId"])
    .withColumn("TotalAmount", col("PricePerUnit") * col("Qty"))
  )

```

Step 5: Create Initial ordersummary Table

The `ordersummary` table is created by joining the cleaned customer and order data.

```

@dlt.table(
    name="ordersummary",
    table_properties={
        "quality": "silver",
        "delta.enableChangeDataFeed": "true"
    },
    comment="SCD Type 2 table tracking customer and order data history",
    temporary=False,
    spark_conf={"spark.databricks.delta.schema.autoMerge.enabled": "true"}
)
def ordersummary():
    current_data = (
        dlt.read("silver_customer")
        .join(
            dlt.read("silver_order"),
            "CustId",
            "inner"
        )
        .select(
            "CustId", "Name", "EmailId", "Region",
            "OrderId", "ItemName", "PricePerUnit", "Qty", "Date"
        )
    )
    try:
        existing_data = dlt.read("ordersummary")
        new_data = (
            current_data
            .withColumn("IsActive", lit(True))
            .withColumn("StartDate", current_timestamp())
            .withColumn("EndDate", lit(None).cast("timestamp"))
        )
        return new_data
    except Exception as e:
        return (
            current_data
            .withColumn("IsActive", lit(True))
            .withColumn("StartDate", current_timestamp())
            .withColumn("EndDate", lit(None).cast("timestamp"))
        )

```

Step 6: Create customeraggregatespend Table

The `customeraggregatespend` table is created by aggregating the total amount spent by each customer.

```

@dlt.table(
    name="customeraggregatespend",
    comment="Aggregated customer spending by name and date",
    temporary=False
)
def customeraggregatespend():
    return (

```

```

dlt.read("silver_order")
.join(dlt.read("silver_customer"), "CustId", "inner")
.groupBy("Name", "Date")
.agg(sum_("TotalAmount").alias("TotalAmount"))
)

```

Step 7: Implement SCD Type 2 Updates for ordersummary Table

The SCD Type 2 updates are implemented by identifying changed records and creating new versions of these records.

```

@dlt.table(
    name="ordersummary_updates",
    comment="Updates for SCD Type 2 implementation",
    temporary=True
)
def ordersummary_updates():
    current_data = (
        dlt.read("silver_customer")
        .join(
            dlt.read("silver_order"),
            "CustId",
            "inner"
        )
        .select(
            "CustId", "Name", "EmailId", "Region",
            "OrderId", "ItemName", "PricePerUnit", "Qty", "Date"
        )
    )

    existing_active = (
        dlt.read("ordersummary")
        .filter(col("IsActive") == True)
    )

    join_condition = (
        (existing_active["CustId"] == current_data["CustId"]) &
        (existing_active["OrderId"] == current_data["OrderId"])
    )

    records_to_expire = (
        existing_active
        .join(
            current_data,
            join_condition,
            "inner"
        )
        .filter(
            (existing_active["Name"] != current_data["Name"]) |
            (existing_active["EmailId"] != current_data["EmailId"])
            |
            (existing_active["Region"] != current_data["Region"])
        )
        .select(
            existing_active["CustId"],
            existing_active["OrderId"]
        )
    )

```

```

    )
    records_to_insert = (
        current_data
        .join(
            records_to_expire,
            ["CustId", "OrderId"],
            "inner"
        )
        .withColumn("IsActive", lit(True))
        .withColumn("StartDate", current_timestamp())
        .withColumn("EndDate", lit(None).cast("timestamp"))
    )
    new_records = (
        current_data
        .join(
            existing_active.select("CustId", "OrderId"),
            ["CustId", "OrderId"],
            "left_anti"
        )
        .withColumn("IsActive", lit(True))
        .withColumn("StartDate", current_timestamp())
        .withColumn("EndDate", lit(None).cast("timestamp"))
    )
    return records_to_insert.union(new_records)

```

Step 8: Apply SCD Type 2 Updates to ordersummary Table

The SCD Type 2 updates are applied by expiring the existing records and inserting the new records.

```

@dlt.table(
    name="apply_ordersummary_updates",
    comment="Apply SCD Type 2 updates to ordersummary table",
    temporary=True
)
def apply_ordersummary_updates():
    existing_active = (
        dlt.read("ordersummary")
        .filter(col("IsActive") == True)
    )
    updates = dlt.read("ordersummary_updates")
    records_to_expire = (
        existing_active
        .join(
            updates.select("CustId", "OrderId"),
            ["CustId", "OrderId"],
            "inner"
        )
    )
    expired_records = (
        records_to_expire
        .withColumn("IsActive", lit(False))
    )

```

```
        .withColumn("EndDate", current_timestamp())  
    )  
    return expired_records.union(updates)
```